

ECE 2011

Introduction to Computer Engineering
Spring 2026

Lecture 15

03-24-2026

Linghao Song

Yale ECE

About Assignment 4, A Bite of Latest Advances in Computer Engineering

- Assignment 4 will be a group presentation project.
- Presentations will take place during the week of April 14.
- Each group will study and present one recent research paper in computer engineering.
- The goal is to learn how to read, understand, and explain a technical paper.

About Assignment 4, Group and Topic Organization

- The class will be divided into groups.
- Two groups will work on the same general topic, but on different papers.
- This will allow us to compare different ideas under the same topic.
- I will provide the list of topics and papers.
- Sign up on the topics you are interested in.

About Assignment 4, What Your Group Will Do

- Read and discuss the assigned paper together.
- Understand the problem, motivation, main idea, and key results.
- Prepare a short presentation slides as a group.
- Prepare a short written summary as a group.
- Everyone should understand the main points of the paper, even if group members take different roles.

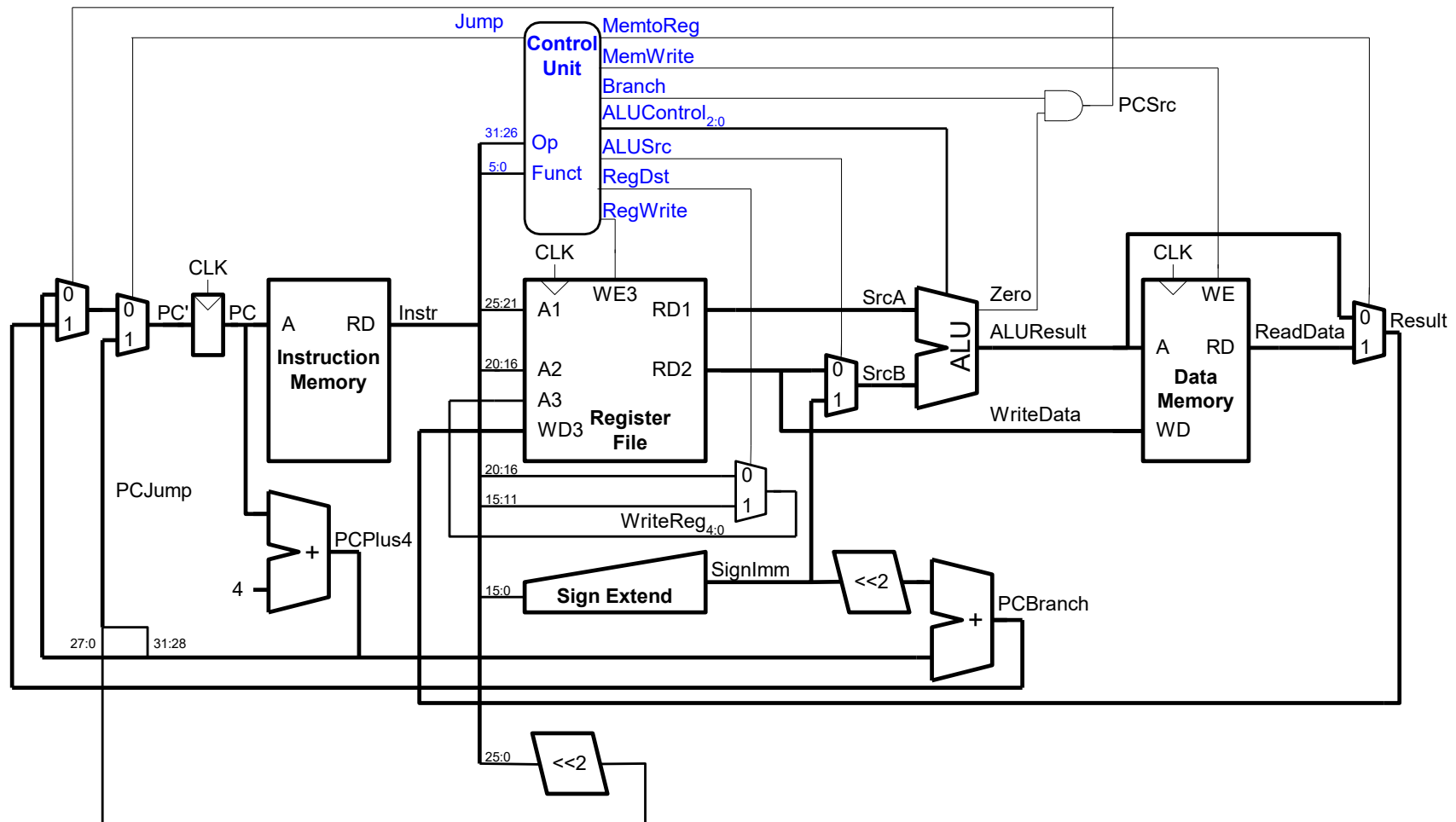
About Assignment 4, Suggested Division of Work

- Group members may divide work in different ways.
- Possible tasks include:
 - reading and understanding the paper
 - preparing slides
 - presenting in class
 - writing the summary
- Even with divided tasks, this should still be a collaborative group effort.

About Assignment 4, Presentation

- In the week of April 14, Tu/Th.
- Your presentation should clearly explain:
 - what problem the paper solves
 - why the problem matters
 - the main idea of the paper
 - how the method or system works at a high level
 - the main results and takeaways
- Focus on clear explanation, not every technical detail.
- More details on groups, topics, papers, and deadlines will be shared soon.

Single-Cycle Processor & Lab7



Assembly Language

- **Instructions:** commands in a computer's language
 - **Assembly language:** human-readable format of instructions
 - **Machine language:** computer-readable format (1's and 0's)
- **MIPS architecture:**
 - Developed by John Hennessy and his colleagues at Stanford and in the 1980's.
 - Used in many commercial systems, including Silicon Graphics, Nintendo, and Cisco

Once you've learned one architecture, it's easy to learn others

Instructions: Addition

C Code

```
a = b + c;
```

MIPS assembly code

```
add a, b, c
```

- **add:** mnemonic indicates operation to perform
- **b, c:** source operands (on which the operation is performed)
- **a:** destination operand (to which the result is written)

Instructions: Subtraction

- Similar to addition - only mnemonic changes

C Code

```
a = b - c;
```

MIPS assembly code

```
sub a, b, c
```

- **sub:** mnemonic
- **b, c:** source operands
- **a:** destination operand

Multiple Instructions

- More complex code is handled by multiple MIPS instructions.

C Code

```
a = b + c - d;
```

MIPS assembly code

```
add t, b, c    # t = b + c  
sub a, t, d    # a = t - d
```

Operands

- Operand location: physical location in computer
 - Registers
 - Memory
 - Constants (also called *immediates*)

Operands: Registers

- MIPS has 32 32-bit registers
- Registers are faster than memory
- MIPS called “32-bit architecture” because it operates on 32-bit data

MIPS Register Set

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

Operands: Registers

- **Registers:**
 - \$ before name
 - Example: \$0, “register zero”, “dollar zero”
- Registers used for **specific purposes:**
 - \$0 always holds the **constant value 0**.
 - the ***saved registers***, \$s0 – \$s7, used to hold variables
 - the ***temporary registers***, \$t0 – \$t9, used to hold intermediate values during a larger computation

Instructions with Registers

- Revisit add instruction

C Code

```
a = b + c
```

MIPS assembly code

```
# $s0 = a, $s1 = b, $s2 = c  
add $s0, $s1, $s2
```

Operands: Memory

- Too much data to fit in only 32 registers
- Store more data in memory
- Memory is large, but slow
- Commonly used variables kept in registers

Reading Memory

- Memory read called **load**

- **Mnemonic:** *load word* (`lw`)

- **Format:**

`lw $s0, 5($t1)`

- **Address calculation:**

- add *base address* (`$t1`) to the *offset* (5)
- `address = ($t1 + 5)`

- **Result:**

- `$s0` holds the value at address (`$t1 + 5`)

addr
each line stores
a different element of the
array `$t0` line0 array[0]

line1 array[1]

?

line6 array[6]

allows you to get the
value at
`$t6`.

Any register may be used as base address

Writing Memory

- Memory write are called **store**
- **Mnemonic:** *store word* (sw)
- **Example:** Write (store) the value in $\$t4$ into memory address 7
 - add the base address ($\$0$) to the offset ($0x7$)
 - address: $(\$0 + 0x7) = 7$Offset can be written in decimal (default) or hexadecimal

Assembly code

```
sw $t4, 0x7($0) # write the value in $t4  
                # to memory word 7
```

destination

Operands: Constants/Immediates

- `lw` and `sw` use constants or *immediates*
- *immediately* available from instruction
- 16-bit two's complement number
- `addi`: add immediate
- Subtract immediate (`subi`) necessary?

C Code

```
a = a + 4;  
b = a - 12;
```

MIPS assembly code

```
# $s0 = a, $s1 = b  
addi $s0, $s0, 4  
addi $s1, $s0, -12
```

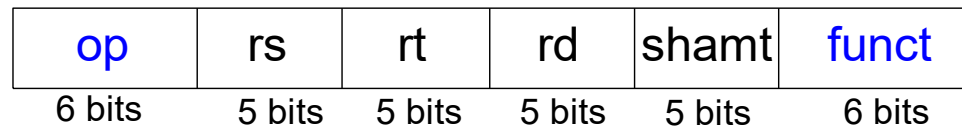
Machine Language

- Binary representation of instructions
- Computers only understand 1's and 0's
- 32-bit instructions
 - Simplicity favors regularity: 32-bit data & instructions
- 3 instruction formats:
 - **R-Type**: register operands
 - **I-Type**: immediate operand
 - **J-Type**: for jumping

R-Type

- *Register-type*
- 3 register operands:
 - rs, rt: source registers
 - rd: destination register
- Other fields:
 - op: the *operation code* or *opcode* (0 for R-type instructions)
 - funct: the *function*
with opcode, tells computer what operation to perform
 - shamt: the *shift amount* for shift instructions, otherwise it's 0

R-Type



R-Type Examples

Assembly Code

```
add $s0, $s1, $s2
```

```
sub $t0, $t3, $t5
```

Field Values

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

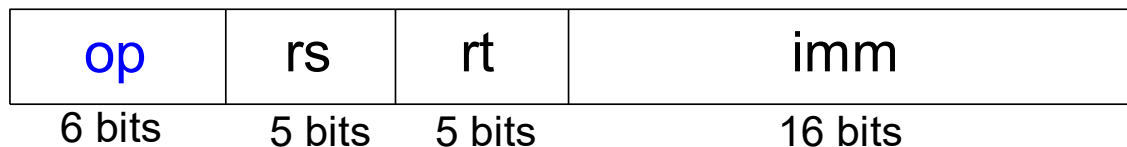
Note the order of registers in the assembly code:

```
add rd, rs, rt
```

I-Type

- *Immediate-type*
- 3 operands:
 - `rs, rt`: register operands
 - `imm`: 16-bit two's complement immediate
- Other fields:
 - `op`: the opcode
 - Simplicity favors regularity: all instructions have opcode
 - Operation is completely determined by opcode

I-Type



I-Type Examples

Assembly Code

Field Values

	op	rs	rt	imm
<code>addi \$s0, \$s1, 5</code>	8	17	16	5
<code>addi \$t0, \$s3, -12</code>	8	19	8	-12
<code>lw \$t2, 32(\$0)</code>	35	0	10	32
<code>sw \$s1, 4(\$t1)</code>	43	9	17	4
	6 bits	5 bits	5 bits	16 bits

Note the differing order of registers in assembly and machine codes:

`addi rt, rs, imm`

`lw rt, imm(rs)`

`sw rt, imm(rs)`

Machine Code

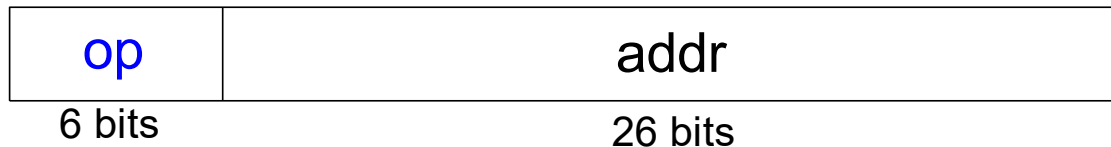
op	rs	rt	imm	
001000	10001	10000	0000 0000 0000 0101	(0x22300005)
001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
6 bits	5 bits	5 bits	16 bits	



Machine Language: J-Type

- *Jump-type*
- 26-bit address operand (`addr`)
- Used for jump instructions (`j`)

J-Type



Review: Instruction Formats

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

J-Type

op	addr
6 bits	26 bits

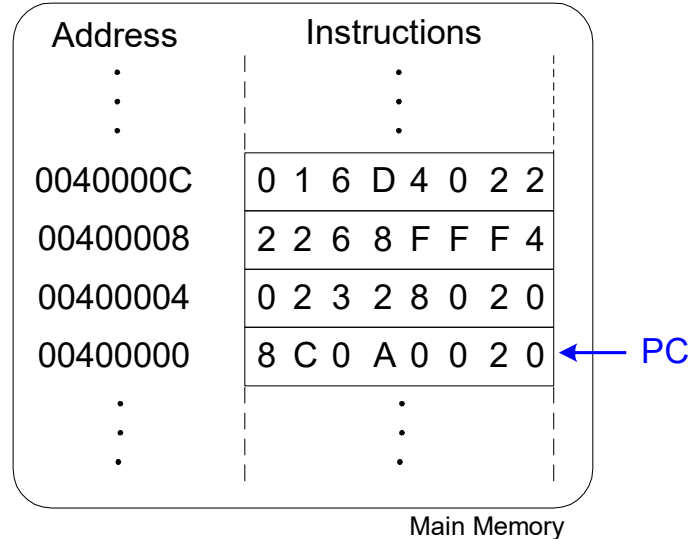
Power of the Stored Program

- 32-bit instructions & data stored in memory
- Sequence of instructions: only difference between two applications
- To run a new program:
 - No rewiring required
 - Simply store new program in memory
- Program Execution:
 - Processor *fetches* (reads) instructions from memory in sequence
 - Processor performs the specified operation

The Stored Program

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022

Stored Program



Program Counter (PC): keeps track of current instruction

Interpreting Machine Code

- Start with opcode: tells how to parse rest
- If opcode all 0's
 - R-type instruction
 - Function bits tell operation
- Otherwise
 - opcode tells operation

Machine Code

Field Values

Assembly Code

op	rs	rt	imm
(0x2237FFF1)	001000	10001	10111 1111 1111 0001
	2	2	3 7 F F F 1

op	rs	rt	imm
	8	17	23 -15

addi \$s7, \$s1, -15

op	rs	rt	rd	shamt	funct
(0x02F34022)	000000	10111	10011	01000	00000 100010
	0	2	F	3	4 0 2 2

op	rs	rt	rd	shamt	funct
	0	23	19	8	0 34

sub \$t0, \$s7, \$s3

Programming

- High-level languages:
 - e.g., C, Java, Python
 - Written at higher level of abstraction
- Common high-level software constructs:
 - if/else statements
 - for loops
 - while loops
 - arrays
 - function calls

Logical Instructions

- **and, or, xor, nor**
 - **and**: useful for **masking** bits
 - Masking all but the least significant byte of a value:
 $0xF234012F \text{ AND } 0x000000FF = 0x0000002F$
 - **or**: useful for **combining** bit fields
 - Combine $0xF2340000$ with $0x000012BC$:
 $0xF2340000 \text{ OR } 0x000012BC = 0xF23412BC$
 - **nor**: useful for **inverting** bits:
 - $A \text{ NOR } \$0 = \text{NOT } A$
- **andi, ori, xori**
 - 16-bit immediate is zero-extended (*not* sign-extended)
 - `nor` not needed

Logical Instructions Example 1

Source Registers

\$s1	1111	1111	1111	1111	0000	0000	0000	0000
\$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code

```
and $s3, $s1, $s2  
or  $s4, $s1, $s2  
xor $s5, $s1, $s2  
nor $s6, $s1, $s2
```

Result

\$s3								
\$s4								
\$s5								
\$s6								

Logical Instructions Example 1

Source Registers

\$s1	1111	1111	1111	1111	0000	0000	0000	0000
\$s2	0100	0110	1010	0001	1111	0000	1011	0111

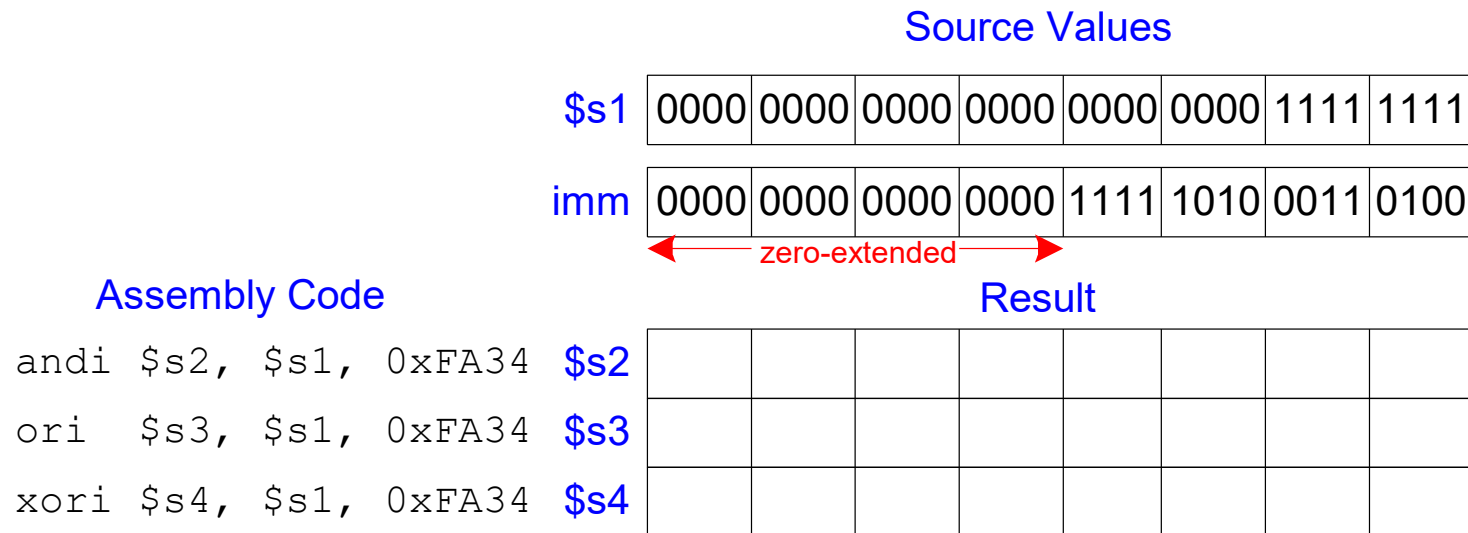
Assembly Code

```
and $s3, $s1, $s2
or  $s4, $s1, $s2
xor $s5, $s1, $s2
nor $s6, $s1, $s2
```

Result

\$s3	0100	0110	1010	0001	0000	0000	0000	0000
\$s4	1111	1111	1111	1111	1111	0000	1011	0111
\$s5	1011	1001	0101	1110	1111	0000	1011	0111
\$s6	0000	0000	0000	0000	0000	1111	0100	1000

Logical Instructions Example 2



Logical Instructions Example 2

		Source Values							
	\$s1	0000	0000	0000	0000	0000	0000	1111	1111
	imm	0000	0000	0000	0000	1111	1010	0011	0100
									
		zero-extended							
Assembly Code		Result							
andi \$s2, \$s1, 0xFA34	\$s2	0000	0000	0000	0000	0000	0000	0011	0100
ori \$s3, \$s1, 0xFA34	\$s3	0000	0000	0000	0000	1111	1010	1111	1111
xori \$s4, \$s1, 0xFA34	\$s4	0000	0000	0000	0000	1111	1010	1100	1011

Shift Instructions

- **sll:** shift left logical
 - **Example:** `sll $t0, $t1, 5` # `$t0 <= $t1 << 5`
- **srl:** shift right logical
 - **Example:** `srl $t0, $t1, 5` # `$t0 <= $t1 >> 5`
- **sra:** shift right arithmetic
 - **Example:** `sra $t0, $t1, 5` # `$t0 <= $t1 >>> 5`

Variable Shift Instructions

- **sllv**: shift left logical variable
 - **Example:** `sllv $t0, $t1, $t2 # $t0 <= $t1 << $t2`
- **srlv**: shift right logical variable
 - **Example:** `srlv $t0, $t1, $t2 # $t0 <= $t1 >> $t2`
- **srav**: shift right arithmetic variable
 - **Example:** `srav $t0, $t1, $t2 # $t0 <= $t1 >>> $t2`

Shift Instructions

Assembly Code

Field Values

	op	rs	rt	rd	shamt	funct
sll \$t0, \$s1, 2	0	0	17	8	2	0
srl \$s2, \$s1, 2	0	0	17	18	2	2
sra \$s3, \$s1, 2	0	0	17	19	2	3
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00010	000000	(0x00114080)
000000	00000	10001	10010	00010	000010	(0x00119082)
000000	00000	10001	10011	00010	000011	(0x00119883)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Branching

- Execute instructions out of sequence
- Types of branches:
 - **Conditional**
 - branch if equal (`beq`)
 - branch if not equal (`bne`)
 - **Unconditional**
 - jump (`j`)
 - jump register (`j r`)
 - jump and link (`j al`)

Review: The Stored Program

Assembly Code

Machine Code

lw	\$t2, 32(\$0)	0x8C0A0020
add	\$s0, \$s1, \$s2	0x02328020
addi	\$t0, \$s3, -12	0x2268FFF4
sub	\$t0, \$t3, \$t5	0x016D4022

Stored Program

Address	Instructions
⋮	⋮
0040000C	0 1 6 D 4 0 2 2
00400008	2 2 6 8 F F F 4
00400004	0 2 3 2 8 0 2 0
00400000	8 C 0 A 0 0 2 0 ← PC
⋮	⋮

Main Memory

Conditional Branching (beq)

MIPS assembly

```
addi $s0, $0, 4      # $s0 = 0 + 4 = 4
addi $s1, $0, 1      # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2      # $s1 = 1 << 2 = 4
beq  $s0, $s1, target # branch is taken
addi $s1, $s1, 1      # not executed
sub  $s1, $s1, $s0     # not executed

target:               # label
add  $s1, $s1, $s0     # $s1 = 4 + 4 = 8
```

Labels indicate instruction location. They can't be reserved words and must be followed by colon (:)

The Branch Not Taken (bne)

MIPS assembly

1	addi	\$s0, \$0, 4	# \$s0 = 0 + 4 = 4
2	addi	\$s1, \$0, 1	# \$s1 = 0 + 1 = 1
3	sll	\$s1, \$s1, 2	# \$s1 = 1 << 2 = 4
4	bne	\$s0, \$s1, target	# branch not taken
5	addi	\$s1, \$s1, 1	# \$s1 = 4 + 1 = 5
6	sub	\$s1, \$s1, \$s0	# \$s1 = 5 - 4 = 1

target:

7	add	\$s1, \$s1, \$s0	# \$s1 = 1 + 4 = 5
---	-----	------------------	--------------------

1 → 2 → 3 → 4 → 5 → 6

Unconditional Branching (j)

MIPS assembly

```
1 addi $s0, $0, 4           # $s0 = 4
2 addi $s1, $0, 1           # $s1 = 1
3 j      target             # jump to target
4 sra    $s1, $s1, 2         # not executed
5 addi    $s1, $s1, 1        # not executed
6 sub     $s1, $s1, $s0      # not executed

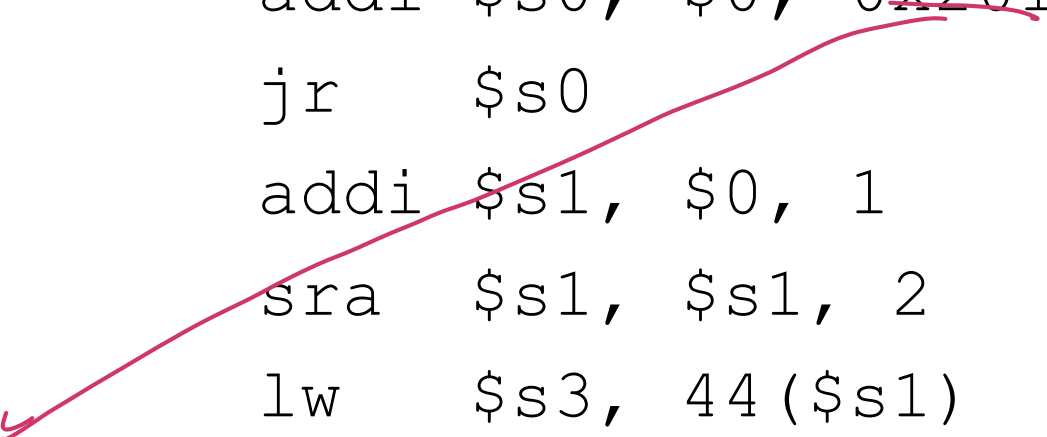
target:
7 add     $s1, $s1, $s0      # $s1 = 1 + 4 = 5
```

1 → 2 → 3 → 7

Unconditional Branching (jr)

MIPS assembly

```
1 0x00002000      addi $s0, $0, 0x2010
2 0x00002004      jr   $s0
      0x00002008      addi $s1, $0, 1
      0x0000200C      sra  $s1, $s1, 2
3 0x00002010      lw   $s3, 44($s1)
```



jr is an **R-type** instruction.

High-Level Code Constructs

- `if` statements
- `if/else` statements
- `while` loops
- `for` loops

If Statement

C Code

```
if (i == j)
    f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

If Statement

C Code

```
if (i == j)
    f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
    bne $s3, $s4, L1
    add $s0, $s1, $s2

L1: sub $s0, $s0, $s3
```

Assembly tests opposite case ($i \neq j$) of high-level code ($i == j$)

If/Else Statement

C Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

If/Else Statement

C Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

```
    bne $s3, $s4, L1
    add $s0, $s1, $s2
```

```
    j done
L1:    sub $s0, $s0, $s3
done:
```

→ unconditional jump.
↓
why needed?

While Loops

C Code

```
// determines the power
// of x such that  $2^x = 128$ 
int pow = 1;
int x    = 0;

while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

MIPS assembly code

While Loops

C Code

```
// determines the power
// of x such that 2x = 128
int pow = 1;
int x    = 0;

while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x

        addi $s0, $0, 1
        add  $s1, $0, $0
        addi $t0, $0, 128
while:   beq  $s0, $t0, done
        sll  $s0, $s0, 1
        addi $s1, $s1, 1
        j    while
done:
```

*multiply by 2 is easy,
you just
left shift.*

Assembly tests for the opposite case (`pow == 128`) of the C code (`pow != 128`).

For Loops

```
for (initialization; condition; loop operation)  
    statement
```

- **initialization:** executes before the loop begins
- **condition:** is tested at the beginning of each iteration
- **loop operation:** executes at the end of each iteration
- **statement:** executes each time the condition is met

For Loops

C Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
    sum = sum + i;
}
```

MIPS assembly code

For Loops

C Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
    sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
addi $s1, $0, 0
add $s0, $0, $0
addi $t0, $0, 10
for: beq $s0, $t0, done
    add $s1, $s1, $s0
    addi $s0, $s0, 1
    j for
done:
```

difference

*honestly my brain stopped
comprehending what's going on at
this pt.*

Less Than Comparison

C Code

```
// add the powers of 2 from 1
// to 100
int sum = 0;
int i;

for (i=1; i < 101; i = i*2) {
    sum = sum + i;
}
```

MIPS assembly code

Less Than Comparison

C Code

```
// add the powers of 2 from 1
// to 100
int sum = 0;
int i;

for (i=1; i < 101; i = i*2) {
    sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
        addi $s1, $0, 0
        addi $s0, $0, 1
        addi $t0, $0, 101
loop:    slt  $t1, $s0, $t0
        beq  $t1, $0, done
        add  $s1, $s1, $s0
        sll  $s0, $s0, 1
        j    loop
done:
```

\$t1 = 1 if $i < 101$

Arrays

- Access large amounts of similar data
- **Index:** access each element
- **Size:** number of elements

Arrays

- 5-element array
- **Base address** = 0x12348000 (address of first element, `array[0]`)
- First step in accessing an array: load base address into a register

0x12340010	array[4]
0x1234800C	array[3]
0x12348008	array[2]
0x12348004	array[1]
0x12348000	array[0]

Accessing Arrays

// C Code

```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

Accessing Arrays

// C Code

```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

MIPS assembly code

\$s0 = array base address

lui	\$s0, 0x1234	# 0x1234 in upper half of \$s0
ori	\$s0, \$s0, 0x8000	# 0x8000 in lower half of \$s0

lw	\$t1, 0(\$s0)	# \$t1 = array[0]
sll	\$t1, \$t1, 1	# \$t1 = \$t1 * 2
sw	\$t1, 0(\$s0)	# array[0] = \$t1

lw	\$t1, 4(\$s0)	# \$t1 = array[1]
sll	\$t1, \$t1, 1	# \$t1 = \$t1 * 2
sw	\$t1, 4(\$s0)	# array[1] = \$t1

→ why did we split??

Arrays using For Loops

// C Code

```
int array[1000];  
int i;  
  
for (i=0; i < 1000; i = i + 1)  
    array[i] = array[i] * 8;
```

MIPS assembly code

```
# $s0 = array base address, $s1 = i
```

Arrays Using For Loops

MIPS assembly code

\$s0 = array base address, \$s1 = i

initialization code

lui \$s0, 0x23B8 # \$s0 = 0x23B80000

ori \$s0, \$s0, 0xF000 # \$s0 = 0x23B8F000

addi \$s1, \$0, 0 # i = 0

addi \$t2, \$0, 1000 # \$t2 = 1000

loop:

slt \$t0, \$s1, \$t2 # i < 1000?

beq \$t0, \$0, done # if not then done

sll \$t0, \$s1, 2 # \$t0 = i * 4 (byte offset)

add \$t0, \$t0, \$s0 # address of array[i]

lw \$t1, 0(\$t0) # \$t1 = array[i]

sll \$t1, \$t1, 3 # \$t1 = array[i] * 8

sw \$t1, 0(\$t0) # array[i] = array[i] * 8

addi \$s1, \$s1, 1 # i = i + 1

j loop # repeat

done: